E-commerce System Test Strategy (MFQ Framework)

1. Objective

Based on the **MFQ (Models-Functional-Quality)** framework, design a systematic test scheme for the e-commerce system. It covers business modeling, functional verification and quality risk control, so as to ensure the consistency, reliability and stability of core order and inventory links.

2. Business Context

Core system modules:

- Commodity: SPU/SKU Management
- Inventory: Status transition of available, reserved, sold and restored stock
- Shopping Cart: Product addition, modification and checkout
- Order Lifecycle: Place Order → Payment → Shipment → Goods Receipt → Return
- External Synchronization: Inventory and order synchronization with WMS/SC2P systems

---

3. M - Models Layer

3.1 End-to-End Business Process Model

`place order → payment → shipment → signed → return`

3.2 Order State Machine Model

Status	Trigger Event	Transition Condition	Target Status
Pending Payment	User completes payment	Payment succeeded	Pending Shipment
Pending Payment	Unpaid upon timeout	Exceed valid payment period	Cancelled
Pending Shipment	System initiates delivery	Successful inventory deduction & WMS synchronization	Pending Receipt
Pending	User confirms	Receipt completed	Completed

Receipt	receipt		
Completed	User submits return request	Return application approved	Returning
Returning	Returned goods inspection done	Inventory recovered & refund finished	Returned / Completed

3.3 Inventory Status Model

Status	Trigger Operation	Status Change
Available	Stock reservation on order placement	→ Reserved
Reserved	Inventory deduction after successful payment	→ DED(Deducted)
Reserved	Order cancellation / Payment timeout	→ Restore to Available
DED(Deducted)	Goods returned and inspected	→ Restore to Available (sales status verification required)

3.4 External Interaction Model

`E-commerce Frontend System ↔ WMS/SC2P`

- Forward Process: Order payment succeeded → Call WMS to deduct inventory → Receive returned result → Update order status
- Reverse Process: WMS inventory changes → Synchronize data to frontend inventory (timed / event-driven)

3.5 Consistency Boundary Model

- **Strong Consistency**: Core links of order placement, payment and inventory deduction (distributed transaction / eventual consistency compensation)
- **Eventual Consistency**: WMS inventory synchronization, asynchronous order status notification
- **Reconciliation & Compensation Points**:
    - Regular inventory reconciliation (frontend inventory vs WMS inventory)
    - Order status reconciliation (frontend orders vs WMS delivery status)
    - Compensation mechanism for message accumulation and loss

4. F - Functional Layer

4.1 Core Happy Path Coverage

1. Regular order placement: User selects goods → Add to cart → Submit order → Payment success → Order status updated
2. Regular payment: Access payment channel → Receive payment success callback → Update payment status → Convert reserved stock to deducted stock
3. Regular shipment: Pending shipment order → Call WMS delivery API → Update order to pending receipt → User confirms receipt
4. Regular receipt: User confirms goods receipt → Order status marked completed
5. Regular return: User submits return application → Application approved → Return goods for inspection → Inventory recovery & refund completion

4.2 Inventory Synchronization Scenarios

Scenario Category	Scenario Description
Successful/Failed Reservation	Reservation succeeds with sufficient stock; fails with insufficient stock
Successful/Failed Deduction	Stock deducted normally after payment; compensation triggered if WMS deduction fails
Cancellation & Recovery	Reserved stock restores available after payment timeout or manual order cancellation
Idempotency Verification	No repeated stock deduction for duplicated payment callbacks and deduction requests
Out-of-order Message Handling	Ensure final inventory consistency when payment notice arrives later than cancellation notice

4.3 Boundary Scenarios

- Zero Inventory: Order placement prohibited when stock is zero; available after stock replenishment
- High-concurrency Flash Sale: Verify no overselling or underselling during simultaneous order submission
- Partial Delivery & Split Order: Partial stock deduction for orders containing multiple commodities
- Return & Inventory Recovery: Stock restored after inspection of returned goods from finished orders

4.4 Key API Verification

- Request field verification: Mandatory item, format, enumeration value and length check
- Error code verification: Return codes for insufficient stock, payment failure, WMS API timeout, unauthorized access and other exceptions
- Callback data verification: Field integrity and idempotency check for payment and WMS delivery callbacks

---

5. Q - Quality Layer

5.1 Performance Indicators

Indicator	Target Value
Concurrency Capacity	Support 1000 QPS order requests
Latency	P95 latency of order API < 200ms, P95 latency of payment callback < 500ms
Error Rate	Error rate of core order and payment APIs < 0.1%
Throughput	Peak order processing capacity ≥ 500 TPS

5.2 Reliability Strategy

- Retry Mechanism: Exponential backoff retry supported for WMS API calls and payment callback processing
- Degradation & Backup: Degrade non-core functions such as product recommendation under high concurrency to guarantee core order process
- Timeout Control: Timeout threshold configured for all external API calls (3 seconds for WMS API)
- Circuit Breaker: Activate circuit protection to avoid system avalanche when WMS service is unavailable
- Compensation Task: Regular inventory reconciliation and order status remediation to fix abnormal data

5.3 Observability

- Logs: Full logs of core links including order placement, payment, deduction, shipment and receipt with traceId

- Metrics: Inventory difference ratio, message backlog volume, stagnant order ratio (orders unchanged over 24 hours)
- Alarm Trigger: Alarms activate when inventory difference ratio > 1%, message backlog > 1000 items, stagnant orders > 100 pieces

5.4 Security Strategy

- Authentication: User identity verification for all APIs; service mutual authentication for WMS access
- Permission Control: Users can only operate personal orders; inventory management requires administrator privileges
- Sensitive Data Desensitization: Mask mobile numbers, payment information and delivery addresses in logs

5.5 Recoverability

- Fault Drill: Simulate WMS breakdown, missing payment callback and inconsistent inventory data to verify compensation mechanism
- Data Restoration Process: Combine manual and automatic methods to handle abnorma inventory and order status

---

6. Exception Handling & Compensation Strategy

1. **Failed WMS API Call**
    - Retry 3 times with exponential backoff. Record abnormal orders and trigger scheduled compensation tasks if retries fail
    - Mark orders as pending compensation and suspend stock deduction to prevent overselling

2. **Lost/Delayed Payment Callback**
    - Regular tasks query order payment status from payment channels and synchronize data to e-commerce system
    - Conduct manual inspection for orders without received callback within 24 hours

3. **Inconsistent Inventory Data**
    - Perform daily inventory reconciliation between frontend and WMS. Trigger alarms when differences exceed threshold
    - Activate compensation tasks to adjust inventory status based on final order information

---

7. Release Access & Exit Criteria

Access Criteria (Pre-release)

- 100% pass rate of core happy path test cases
- 100% pass rate of inventory reservation, deduction and recovery test cases
- No overselling or underselling in high-concurrency pressure test; all performance indicators achieved
- No high-risk vulnerabilities detected in security scan

Exit Criteria (Online Operation)

- Error rate of core APIs $< 0.1\%$
- Inventory difference ratio $< 0.1\%$
- No blocking online faults; 100% alarm disposal completion rate

---

8. Priority & Trade-off

- **High Priority**: Order and inventory consistency, idempotency of payment and WMS callbacks, overselling and underselling risks
- **Medium Priority**: Performance indicators, exception compensation mechanism, log and monitoring coverage
- **Low Priority**: Rare non-core boundary scenarios such as extreme out-of-order message combinations, covered by online monitoring and post-processing compensation